

Byte the Bullet — Code Walkthrough

A beginner's guide to every tag and element used in `byte_the_bullet.html`

This file goes with `byte_the_bullet.html`. It explains, from scratch, every HTML tag, every CSS idea, and every JavaScript building block used to make the live quiz work — so you can open the code and actually understand what each line is doing.

1. HTML — the page's skeleton

HTML (HyperText Markup Language) describes the structure of a page using tags. A tag is a word inside angle brackets, like `<div>`. Most tags come in pairs: an opening tag and a closing tag with a slash, like `<div>...</div>`, wrapping the content that belongs to it.

```
<!doctype html>
```

The very first line of any modern HTML file. It tells the browser 'read this using the current HTML5 rules.' Without it, old browsers guess and can render things oddly.

```
<html lang="en">
```

The root tag — everything else lives inside it. `lang="en"` tells screen readers and search engines the page is in English.

```
<head>
```

A container for information ABOUT the page that isn't shown directly on screen — things like the title, fonts, and page settings.

```
<meta charset="UTF-8">
```

Tells the browser which set of characters/symbols to expect, so emoji (🐼) and special characters display correctly instead of turning into garbled boxes.

```
<meta name="viewport" ...>
```

Controls how the page scales on phones. Our value fixes the zoom level so the quiz looks the same size on every team's phone and doesn't let someone accidentally pinch-zoom mid-question.

```
<title>
```

The text shown in the browser tab. Here it's just 'Byte the Bullet'.

```
<style>
```

Everything between this tag and `</style>` is CSS — the rules that control colors, spacing, fonts and animation (see Section 2).

```
<body>
```

Everything visible on the page goes inside `<body>`. In this file, `body` contains almost nothing directly — just one empty `<div id="app"></div>` — because the whole quiz is drawn dynamically by JavaScript (Section 3).

```
<div>
```

A generic invisible box with no special meaning of its own — used purely to group things together so CSS can style them or JavaScript can find them. The entire quiz UI is built from nested `<div>`s: the card, the timer bar, the leaderboard rows, and so on.

```
<h1>, <h2>, <h3>
```

Headings, in order of importance (`h1` = biggest/most important). Used for the 'Byte the Bullet' brand name and section titles like 'Team Name'.

```
<p>
```

A paragraph of text — used for instructions and small captions like 'Waiting for teams to join...'.

<button>

A clickable button. Every action in the app — Host a Game, Join a Game, Start Game, an answer choice, Next Question — is a <button>, because buttons are automatically focusable and tappable/clickable on both desktop and mobile.

<input type="text">

A single-line text box the user can type into — used for the room code and the team name.

Like <div>, but for small pieces of text inside a line (inline), rather than a whole block. Used for little colored dots and inline labels.

id="..." attribute

A unique name tag on an element, e.g. id="app". JavaScript uses document.getElementById('app') to find that exact element and change what's inside it.

class="..." attribute

One or more style labels on an element, e.g. class="opt selected". CSS rules target classes (Section 2) to decide how things look. An element can have several classes at once, separated by spaces.

data-i="..." attribute

A 'data attribute' — a custom label you invent yourself to store a small piece of information on an element. Here, each answer button stores which option number (0, 1, 2 or 3) it is, so JavaScript can read it back with btn.dataset.i when tapped.

2. CSS — the page's look and feel

CSS (Cascading Style Sheets) is a list of rules shaped like: `selector { property: value; }`. The selector says WHICH elements the rule applies to; the curly braces hold the styling.

:root and --variables

The `:root` block defines custom variables such as `--chili: #C0432D`. Anywhere else in the CSS, `var(--chili)` reuses that exact color. This means the whole quiz's color palette is defined once at the top and reused everywhere, so changing one line changes the whole theme.

Selectors: tag, .class, #id

A bare word like `body` targets every `<body>` tag. A dot like `.card` targets every element with `class="card"`. A hash like `#app` targets the one element with `id="app"`. Combined selectors like `.opt.correct` target elements that have BOTH classes.

box-sizing: border-box

Makes padding and borders count INSIDE an element's declared width, instead of adding extra size on top — this avoids a lot of confusing layout bugs, so it's set once on every element with the `*` (universal) selector.

display: flex

Turns an element into a 'flexbox' — a container that lines its children up in a row (or column) and makes spacing between them easy. Used for the Host/Join buttons sitting side by side, and for centering things.

flex-direction / align-items / justify-content / gap

Flexbox helper properties: `direction` chooses row vs column, `align-items` controls cross-axis alignment, `justify-content` controls main-axis spacing, and `gap` adds even spacing between children without needing extra margin rules.

border-radius

Rounds the corners of a box. A small value like `10px` gives soft rounded corners; `50%` turns a square into a circle (used for the little pulsing dot and the round chili '•' icon).

transition

Tells the browser to animate a property change smoothly instead of jumping instantly. For example `transition: width 1.1s` makes a leaderboard bar visibly grow over about a second when its width changes, instead of snapping straight to full size.

@keyframes and animation

`@keyframes` defines a named sequence of style 'snapshots' at different percentages of time (0%, 50%, 100%...). The `animation` property then plays that sequence on an element. This is how the rocket flies up and fades (`rocketFly`), the ice cream bounces up and down (`iceBounce`), and the confetti pieces fall and spin (`confettiFall`).

transform

Moves, rotates, or resizes an element visually without affecting the layout around it. `translateY` moves something up/down, `rotate` spins it, `scale` grows or shrinks it — combined inside the keyframes above to make the celebration animations feel physical.

opacity

Controls transparency from 0 (invisible) to 1 (fully visible). Animations fade elements out by animating opacity down to 0 at the same time they move away.

position: relative / absolute

By default elements stack top-to-bottom. position: relative lets an element be a positioning 'anchor' for its children. position: absolute takes a child out of the normal flow and places it precisely (using left/top/right/bottom) relative to that anchor — used to float the rocket, ice cream, and confetti pieces on top of the final results card.

Pseudo-class :disabled / :active / :focus

Styles that only apply in a certain state — button:disabled dims a button that's been switched off in JavaScript (e.g. after a team has already answered), and :active gives a little 'press' shrink effect on tap.

@import (Google Fonts)

Pulls in two extra typefaces — Space Grotesk for headings/numbers and Inter for body text — from Google's font service, so the quiz doesn't rely only on generic system fonts.

3. JavaScript — the page's brain

JavaScript is the programming language that makes the page interactive: it reacts to clicks, changes what's on screen, and talks to the shared storage that keeps every device in sync.

```
&lt;script&gt; ... (function(){ ... })();
```

Everything runs inside an 'immediately invoked function expression' — a function defined and called in the same breath. This is a common trick to keep all of the app's variables private, so they can't accidentally clash with anything else on the page.

const / let

Ways to create a variable (a named box that holds a value). `const` means the box's contents shouldn't be reassigned later (e.g. `const ROUNDS = [...]`); `let` is used when a value needs to change over time (e.g. `let pos = 0`).

Arrays [] and objects { }

An array is an ordered list, like the 4 answer options for a question: `['Alan Turing', 'John McCarthy', ...]`. An object is a set of named fields, like `{ score: 0 }` or `{ name, choice, ts, correct }` — the quiz's entire question bank and game state are built from arrays and objects nested inside each other.

Arrow functions (a, b) => { ... }

A short way to write a function. `(name) => toyFor(name)` means 'a function that takes `name` and returns `toyFor(name)`'. Used constantly for small one-off pieces of logic, like sorting or mapping over lists.

`.map()`, `.filter()`, `.sort()`, `.forEach()`

Built-in array tools. `.map()` transforms every item into something new (e.g. turning each question into a line of HTML). `.filter()` keeps only items that pass a test (e.g. only the correct answers). `.sort()` reorders a list (e.g. fastest correct answer first). `.forEach()` just runs some code once per item, without building a new list.

Template literals ``text ${variable}``

Backtick-quoted strings that can embed variables directly inside `${ }` — this is how the code builds chunks of HTML text mixed with live data, like `<h1>${teamName}</h1>`.

async / await

'`async`' marks a function as one that might have to wait for something slow (like fetching data). '`await`' pauses that function until the slow thing finishes, without freezing the whole page. Almost every storage read/write in this app is awaited, because talking to shared storage takes a moment.

Promises and `try/catch`

An `async` operation returns a 'promise' — a stand-in for a value that will exist later. `try/catch` lets the code attempt something risky (like reading a key that might not exist yet) and quietly handle the failure instead of crashing the page.

`fetch(url, options)`

The browser's built-in tool for talking to another website/server over the internet. This app uses it to call `kvdb.io` — a free, public 'key-value' database — so every device (host and every team's phone) can read and write the same shared game data even when the page is hosted somewhere plain like GitHub Pages, which has no database of its own.

kvdb.io buckets and keys

A 'bucket' is a private little database created fresh each time you tap Host a Game (one POST request to kvdb.io returns a random bucket ID). A 'key' is a named slot inside that bucket, like meta or team:Ctrl+Alt+Elite. `fetch(...,{method:'PUT'})` writes a key, a plain `fetch(...)` reads one back, and requesting a URL ending in `?prefix=ans:` lists every key that starts that way — used to gather every team's answer to the current question.

URLSearchParams

A built-in helper for reading the `?key=value` part of a web address. When a student opens a join link like `index.html?b=abc123`, `new URLSearchParams(window.location.search).get('b')` pulls out `abc123` — the bucket ID — automatically, so they don't have to type anything to land in the right room.

JSON.stringify / JSON.parse

Storage only holds plain text, but the app's data is full of objects and arrays. `JSON.stringify` turns a JavaScript object into text before saving it; `JSON.parse` turns that text back into a usable object after reading it.

setInterval / clearInterval

`setInterval(fn, 900)` repeatedly runs a function every 900 milliseconds — this is literally the 'polling' that makes the quiz feel live: every device keeps re-checking shared storage roughly once a second to see if the host has moved on. `clearInterval` stops that repeating check, e.g. when leaving a screen.

document.getElementById(...)

Finds the one element on the page with a matching id, so its contents or properties can be changed from JavaScript — for example, updating the roster list or disabling the Start button.

element.innerHTML

Replaces everything inside an element with new HTML content. The whole app works by rebuilding `app.innerHTML` every time the game state changes, which is why the UI can look completely different from one screen to the next even though there's only one `<div id="app">` in the actual HTML.

addEventListener / .onclick

Attaches a function to run when something happens — most often a click. `button.onclick = async () => { ... }` means 'when this button is clicked, run this code.'

Date.now()

Returns the current time as a number (milliseconds since 1970). Used to timestamp when a question started and when a team answered, so the code can calculate the 10-second countdown and figure out who answered fastest.

Math.max / Math.min / Math.round / Math.random

Standard math helpers: `max/min` pick the bigger/smaller of two numbers (used to stop a countdown going below 0), `round` removes decimal places (for score numbers), and `random()` produces an unpredictable number between 0 and 1 (used to generate the room code letters and confetti positions).

String methods: .trim(), .toUpperCase(), .replace()

`.trim()` removes stray spaces a user might type by accident. `.toUpperCase()` forces the room code to always be capital letters. `.replace()` is used in the `escapeHtml()` helper to swap dangerous characters like `<` for their

safe text equivalents, so a team typing something odd as their name can't break the page.

4. How it all fits together

There is only ONE html file, and only one real `<div id="app">` box in it. Every screen you see — landing page, host lobby, a live question, the leaderboard, the final podium — is just JavaScript replacing what's inside that one box with new HTML text, built from the ROUNDS/FLAT data at the top of the script.

The 'multiplayer' feeling comes from every device — host and every team's phone — repeatedly fetching the same few keys from the shared kvdb.io bucket (the room's meta info, the roster, each team's score, and each team's answer) roughly once a second, and redrawing the screen if something changed. Nobody is directly connected to anybody else; everyone is just polling the same shared online notebook.

Tip: open `byte_the_bullet.html` in a text editor next to this PDF, find a piece of code you don't recognise, and search this document for it — most things used in the file are explained above.